

BÖLÜM 3

Sözdizimi ve Anlambilim Açıklaması

Describing Syntax and Semantics

Robert W. Sebesta | Concepts of Programming Languages, 12. Baskı

İÇİNDEKİLER

- 3.1 Giriş — Sözdizimi ve Anlambilim Tanımları
- 3.2 Sözdiziminin Genel Problemi — Lexeme, Token, Tanıyıcı, Üretici
- 3.3 Formal Yöntemler — BNF, EBNF, Gramer, Parse Tree, Muğlaklık
- 3.4 Öznitelik Gramerleri — Statik Anlambilim, Tip Kontrolü
- 3.5 Dinamik Anlambilim — Operasyonel, Denotasyonel, Aksiyomatik
- Q Quiz Soruları ve Cevapları
- P Kitap Bölüm Sonu Soruları ve Çözümleri

3.1 Giriş — Sözdizimi ve Anlambilim

Bir programlama dilini tanımlamak, dilin başarısı için kritik öneme sahiptir. Dil tasarımcıları, derleyici yazarları ve dil kullanıcıları — hepsi dili farklı amaçlarla anlamak ister. Temel iki kavram:

Kavram	İngilizce Terimi	Tanım	Örnek
Sözdizimi	Syntax	İfadelerin, deyimlerin biçimi	while (bool_expr) statement
Anlambilim	Semantics	İfadelerin, deyimlerin anlamı	Boolean true iken gövde çalışır, tekrar kontrol edilir

► Neden Önemli?

Sözdizimini tanımlamak anlambilimini tanımlamaktan daha kolaydır, çünkü sözdizimi için BNF gibi evrensel bir notasyon mevcuttur. Anlambilim için böyle bir standart henüz geliştirilmemiştir.

► Dilin Hedef Kitleleri

- İlk değerlendiriciler (Initial Evaluators): Tasarım aşamasında inceleyen potansiyel kullanıcılar.
- Uygulayıcılar (Implementors): Derleyici/yorumlayıcı yazarları; tam ve kesin tanıma ihtiyaç duyarlar.
- Kullanıcılar (Users): Referans kılavuzuna bakarak kod yazan programcılar.

3.2 Sözdizimini Tanımlamanın Genel Problemi

3.2.1 Temel Kavramlar: Lexeme, Token, Cümle

Bir dil, bir alfabenin karakter dizilerinin kümesidir. Bu dizilere cümle (sentence) veya deyim (statement) denir. Formal sözdizimi tanımları genellikle en alt düzey sözdizimsel birimlerden — lexeme'lerden — başlar.

Kavram	İngilizce	Açıklama	Örnek
Sözcükbirim	Lexeme	Programın en küçük anlamlı birimi	index, =, 2, *, +, 17, ;
Belirteç	Token	Lexeme grubunun kategori adı	identifier, equal_sign, int_literal
Tanıyıcı	Recognizer	Dizinin dile ait olup olmadığını söyler	Derleyici sözdizimi analizörü
Üretici	Generator	Dilin cümlelerini üreten cihaz	Gramerler (BNF)

Java Deyiminin Lexeme-Token Analizi: `index = 2 * count + 17;`

Lexeme-Token Eşleştirmesi

Lexeme Token

index identifier

= equal_sign

2 int_literal

* mult_op

count identifier

+ plus_op

17 int_literal

; semicolon

3.2.2 Dil Tanımlama Yaklaşımları

► Tanıyıcı (Recognizer)

Verilen bir dizeyi alır ve dilin üyesi olup olmadığını söyler. Derleyicinin sözdizimi analiz bölümü (parser) bir tanıyıcıdır. Tüm cümleleri değil, yalnızca verilen programın doğru olup olmadığını belirler.

► Üretici (Generator)

Dilin cümlelerini üreten bir cihazdır. Gramerler bu kategoridedir. Kullanıcılar tarafından tanıyıcıdan daha kolay anlaşılır; doğru sözdizimi doğrudan görülür. Her context-free gramer için algoritmik olarak bir tanıyıcı inşa edilebilir.

3.3 Formal Yöntemler — BNF ve Context-Free Grammars

3.3.1 BNF Tarihi ve Context-Free Grammars

Araştırmacı	Katkı
Noam Chomsky (1956)	4 gramer sınıfı tanımladı: regular, context-free, context-sensitive, unrestricted. Programlama dilleri için context-free ve regular sınıfları kullanılır.
John Backus (1959)	ALGOL 58 için formal sözdizimi notasyonu geliştirdi (BNF'nin ilk hali).
Peter Naur (1960)	ALGOL 60 tanımı için BNF'yi düzenleyerek yaygın formunu oluşturdu.

3.3.2 BNF Temelleri

► BNF (Backus-Naur Form) Nedir?

BNF, programlama dilleri için bir üst dil (metalanguage)dir. Context-free gramerlere neredeyse özdeştir. Sözdizimini tanımlamanın en popüler yöntemi olmaya devam etmektedir.

Bileşen	Açıklama	Örnek
Nonterminal	Soyut sözdizimi kategorisi; <> ile gösterilir	<assign>, <expr>, <var>
Terminal	Gerçek sözcükbirim veya belirteç	=, +, begin, end, A, B, C
Kural (Production)	LHS → RHS; bir nonterminalin tanımı	<assign> → <var> = <expression>
(OR)	Birden fazla alternatif tanım	<var> → A B C
Başlangıç Sembolü	Türetmenin başladığı nokta	<program>

BNF Örnekleri

Atama ifadesi:

<assign> → <var> = <expression>

Örnek cümle: total = subtotal1 + subtotal2

Java if-else:

<if_stmt> → if (<logic_expr>) <stmt>

| if (<logic_expr>) <stmt> else <stmt>

3.3.3 Listeler ve Özyineleme (Recursion)

BNF'de değişken uzunluklu listeler için özyineleme kullanılır. Bir kural, LHS'i kendi RHS'inde içeriyorsa özyinelemeli (recursive)dir.

Özyinelemeli Liste Tanımı

Tanımlayıcı listesi:

<ident_list> → identifier

| identifier , <ident_list>

Üretilen örnekler: x | x, y | x, y, z | ...

3.3.4 Türetme (Derivation) ve Sentential Form

Bir gramer, başlangıç sembolünden başlayarak kuralların ardışık uygulanmasıyla cümle üretir. Bu sürece türetme (derivation) denir. Türetmedeki her ara diziye sentential form adı verilir. Son satırda yalnızca terminaller bulunur — bu bir cümle (sentence)dir.

► Soldan Türetme (Leftmost Derivation)

Her adımda en soldaki nonterminal değiştirilir. Türetmenin yönü (soldan ya da sağdan) üretilen dili etkilemez; yalnızca gösterim biçimini değiştirir.

Örnek 3.1 Grameri:

Örnek 3.1: Küçük Dil Grameri

<program> → begin <stmt_list> end

<stmt_list> → <stmt>

| <stmt> ; <stmt_list>

<stmt> → <var> = <expression>

<var> → A | B | C

<expression> → <var> + <var> | <var> - <var> | <var>

begin A = B + C ; B = C end Türetimi:

Soldan Türetme

```
<program> => begin <stmt_list> end
=> begin <stmt> ; <stmt_list> end
=> begin <var> = <expression> ; <stmt_list> end
=> begin A = <expression> ; <stmt_list> end
=> begin A = <var> + <var> ; <stmt_list> end
=> begin A = B + C ; <stmt_list> end
=> begin A = B + C ; <stmt> end
=> begin A = B + C ; B = C end ← CÜMLE
```

3.3.5 Parse Tree (Ayrıştırma Ağacı)

Her iç düğüm (internal node) bir nonterminal ile, her yaprak (leaf) bir terminal ile etiketlenir. Her alt ağaç bir soyutlamanın örneğini tanımlar.

Seviye	Düğüm / Çocuklar	Açıklama
Kök	<assign>	Tüm atama ifadesi
1.	<id> A = <expr>	Sol taraf, eşittir, sağ taraf
2. (<expr>)	<id> B * <expr>	Çarpma: B * (...)
3. (iç <expr>)	(<expr>)	Parantezli ifade
4.	<id> A + <id> C	Toplama: A + C
Yapraklar	A, =, B, *, (, A, +, C,)	Yalnızca terminaller

3.3.6 Muğlaklık (Ambiguity)

Bir gramer, bir cümle için iki veya daha fazla farklı parse tree üretiyorsa muğlak (ambiguous)tır. Derleyiciler anlambilimi parse tree'den türettiği için bu ciddi bir problemdir.

► Muğlaklık Neden Tehlikelidir?

Farklı parse tree'ler farklı anlam ve farklı makine kodu üretir. Örneğin 'A = B + C * A' ifadesinde: bir tree'de toplama önce, diğerinde çarpma önce hesaplanır. Derleyici doğru kodu üretemez.

Muğlak Gramer Örneği

Muğlak Gramer (Örnek 3.3):

<assign> → <id> = <expr>

<id> → A | B | C

```
<expr> → <expr> + <expr> # Her iki yönde büyüyebilir!
```

```
| <expr> * <expr> # ← muğlaklık kaynağı
```

```
| ( <expr> )
```

```
| <id>
```

```
# A = B + C * A için iki ayrı parse tree:
```

```
# Parse Tree 1: + köke yakın → + önce hesaplanır
```

```
# Parse Tree 2: * köke yakın → * önce hesaplanır
```

3.3.7 Operatör Önceliği (Operator Precedence)

Parse tree'de bir operatör ne kadar aşağıda üretilirse, o kadar yüksek önceliğe sahiptir (önce değerlendirilir). Muğlak olmayan bir gramer ile öncelik belirtilir.

Örnek 3.4: Öncelik Belirten Gramer

```
# Örnek 3.4 — Öncelik Gösteren Muğlak Olmayan Gramer:
```

```
<assign> → <id> = <expr>
```

```
<id> → A | B | C
```

```
<expr> → <expr> + <term> # + üstte = düşük öncelik
```

```
| <term>
```

```
<term> → <term> * <factor> # * ortada = yüksek öncelik
```

```
| <factor>
```

```
<factor> → ( <expr> )
```

```
| <id>
```

```
# * her zaman + 'dan daha derin → her zaman önce hesaplanır
```

```
# A = B + C * A türetimi:
```

```
<assign> => A = <expr> => A = <expr> + <term>
```

```
=> A = <term> + <term> => A = B + <term> * <factor>
```

```
=> A = B + C * A
```

3.3.8 Operatör Birleşimi (Associativity)

Aynı önceliğe sahip iki operatör yan yana geldiğinde ($A / B * C$), hangisinin önce değerlendirileceği birleşim (associativity) kuralıyla belirlenir.

Birleşim Türü	Gramer Özelliği	Örnek Kural	Sonuç
---------------	-----------------	-------------	-------

Sol Birleşim	LHS, RHS'nin başında tekrarlanır	$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$	$(A + B) + C$
Sağ Birleşim	LHS, RHS'nin sonunda tekrarlanır	$\langle \text{factor} \rangle \rightarrow \langle \text{exp} \rangle ** \langle \text{factor} \rangle$	$A ** (B ** C)$

► Neden Kritik?

Toplama matematiksel olarak birleşimlidir ancak floating-point toplama bilgisayarda birleşimli değildir. Çıkarma ve bölme hiçbir zaman birleşimli değildir. Bu nedenle doğru birleşim belirtmek kritik önem taşır.

3.3.9 if-else Muğlaklığı (Dangling Else)

İç içe if-else yapısında else hangi if'e ait olduğu belirsiz olabilir. Çözüm: matched ve unmatched deyimler arasında ayırım yapmak.

if-else Muğlaklığı ve Çözümü

```
# Muğlak yapı (dangling else):

if (done == true)
if (denom == 0)
quotient = 0;
else quotient = num / denom; # Hangi if'e ait?

# Çözüm: matched/unmatched ayrımı

<stmt> → <matched> | <unmatched>

<matched> → if (<logic_expr>) <matched> else <matched>
| herhangi non-if deyimi

<unmatched> → if (<logic_expr>) <stmt>
| if (<logic_expr>) <matched> else <unmatched>

# Kural: else, en yakın eşleşmemiş then ile eşleşir.
```

3.3.10 Extended BNF (EBNF)

EBNF, BNF'nin tanımlayıcı gücünü artırmadan okunabilirliğini ve yazılabilirliğini geliştiren üç temel uzantı içerir.

Uzantı	Notasyon	Anlam	Örnek
Opsiyonel	[...]	0 veya 1 kez	$\langle \text{if_stmt} \rangle \rightarrow \text{if} (\langle \text{expr} \rangle) \langle \text{stmt} \rangle [\text{else} \langle \text{stmt} \rangle]$
Tekrar	{ ... }	0 veya daha fazla kez	$\langle \text{ident_list} \rangle \rightarrow \langle \text{id} \rangle \{ , \langle \text{id} \rangle \}$

Seçim	(A B)	Birini seç	$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle (* / \%) \langle \text{factor} \rangle$
-------	-----------	------------	--

BNF ile EBNF Karşılaştırması

BNF:

$\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \mid \langle \text{expr} \rangle - \langle \text{term} \rangle \mid \langle \text{term} \rangle$

$\langle \text{term} \rangle \rightarrow \langle \text{term} \rangle * \langle \text{factor} \rangle \mid \langle \text{term} \rangle / \langle \text{factor} \rangle \mid \langle \text{factor} \rangle$

EBNF (Örnek 3.5 — daha kısa ve okunabilir):

$\langle \text{expr} \rangle \rightarrow \langle \text{term} \rangle \{ (+ \mid -) \langle \text{term} \rangle \}$

$\langle \text{term} \rangle \rightarrow \langle \text{factor} \rangle \{ (* \mid /) \langle \text{factor} \rangle \}$

$\langle \text{factor} \rangle \rightarrow \langle \text{exp} \rangle \{ ** \langle \text{exp} \rangle \}$

$\langle \text{exp} \rangle \rightarrow (\langle \text{expr} \rangle) \mid \text{id}$

► Dikkat: EBNF ve Birleşim

BNF'deki ' $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle$ ' sol özyinelemeli olduğu için sol birleşimi açıkça belirtir. Ancak EBNF'deki ' $\{ + \langle \text{term} \rangle \}$ ' birleşim yönünü belirtmez. Bu durumda syntax analizörü birleşim yönünü kendi mantığıyla uygulamak zorundadır.

3.4 Öznitelik Gramerleri (Attribute Grammars)

3.4.1 Statik Anlambilim (Static Semantics)

BNF bazı dil kurallarını tanımlamakta yetersiz kalır. Bu kurallar statik anlambilim (static semantics) kapsamına girer; derleme zamanında kontrol edilebildikleri için bu adı alırlar.

Durum	Örnek	BNF ile Durum
Zor ama mümkün	Tip uyumluluk kuralları ($\text{int} \neq \text{float}$ ataması)	Mümkün, ancak gramer çok büyük
İmkânsız	Değişken kullanımdan önce tanımlanmalı	Matematiksel olarak BNF ile tanımlanamaz

3.4.2 Öznitelik Gramerinin Yapısı

► Öznitelik Grameri Bileşenleri (Knuth, 1968)

- Öznitelikler (Attributes): Gramer sembollerine atanan değişken benzeri özellikler
- Anlambilim Fonksiyonları (Semantic Functions): Öznitelik değerlerini hesaplar
- Yüklem Fonksiyonları (Predicate Functions): Statik anlambilim kurallarını doğrular

Öznitelik Türü	İngilizce	Bilgi Akışı	Hesaplama
Sentezlenmiş	Synthesized	Alt → Üst (yukarı)	$S(X_0) = f(A(X_1), \dots, A(X_n))$
Kalıtılan	Inherited	Üst/kardeş → Alt (aşağı)	$I(X_j) = f(A(X_0), \dots, A(X_{j-1}))$
İçsel	Intrinsic	Parse tree dışından	Yaprak düğümlerde başlangıç değeri

3.4.3 Tip Kontrolü Örneği (Örnek 3.6)

Değişkenler A, B, C; tipler int veya real. İki değişkende farklı tipler olabilir ($\text{int} + \text{real} \rightarrow \text{real}$). Atama: sol taraf ile sağ taraf tipi eşit olmalı.

Örnek 3.6: Öznitelik Grameri

```
# Sözdizimi:

<assign> → <var> = <expr>

<expr> → <var>[2] + <var>[3] | <var>

<var> → A | B | C

# Kural 1: <assign> → <var> = <expr>

# Semantik: <expr>.expected_type ← <var>.actual_type

# Kural 2: <expr> → <var>[2] + <var>[3]
```

```

# Semantik: <expr>.actual_type ←
# if (var[2].actual_type=int AND var[3].actual_type=int) then int else real
# Yükleme: <expr>.actual_type == <expr>.expected_type

# Kural 3: <expr> → <var>
# Semantik: <expr>.actual_type ← <var>.actual_type
# Yükleme: <expr>.actual_type == <expr>.expected_type

# Kural 4: <var> → A | B | C
# Semantik: <var>.actual_type ← look-up(<var>.string)

```

Adım	İşlem	Kural
1	<var>.actual_type ← look-up(A) = real	Kural 4
2	<expr>.expected_type ← real	Kural 1
3	<var>[2].actual_type ← real; <var>[3].actual_type ← int	Kural 4
4	<expr>.actual_type ← real (int + real → real)	Kural 2
5	Yükleme: real == real → TRUE ✓	Kural 2

3.5 Dinamik Anlambilim — Program Anlamının Açıklanması

Dinamik anlambilim, ifadelerin ve deyimlerin anlam ve etkilerini tanımlar. Evrensel kabul görmüş tek bir notasyon yoktur. Üç temel yaklaşım:

Yaklaşım	Temel Fikir	Mat. Temel	Güçlü Yönü
Operasyonel	İdeal makine üzerinde çalıştırarak tanımla	Programlama dilleri	Anlaşılır; dil kullanıcıları için iyi
Denotasyonel	Matematiksel nesnelere dönüştür	Özyinelemeli fonksiyon teorisi	En titiz ve kapsamlı yöntem
Aksiyomatik	Mantık ifadeleriyle doğruluğu ispat et	Predicate Calculus	Program doğrulama araştırması

3.5.1 Operasyonel Anlambilim (Operational Semantics)

Temel fikir: Bir deyimın anlamını, onu ideal bir makine üzerinde çalıştırmanın etkileriyle tanımlamak. Makine durumu, depolama alanındaki değerlerin toplamıdır.

Düzy	Ad	Odak
Üst	Doğal Operasyonel Semantik	Programın tüm nihai sonucu
Alt	Yapısal Operasyonel Semantik	Her durum değişikliğinin tam dizisi

for Döngüsü — Operasyonel Semantik

C 'for' döngüsünün operasyonel semantiği:

C Deyimi Anlam (Ara Dil)

```
for (expr1; expr2; expr3) { expr1;
... loop: if expr2 == 0 goto out
} ...
expr3;
goto loop
out: ...
```

► Sınırlılıkları

- Matematiksel değil, programlama diline dayalıdır → dairesel tanımlar riski
- Gerçek makine dili adımları çok küçük ve sayısızdır
- Gerçek bilgisayar belleği çok büyük ve karmaşıktır

3.5.2 Denotasyonel Anlambilim (Denotational Semantics)

En titiz formal yöntemdir. Her dil varlığı için bir matematiksel nesne ve bu nesneye eşleme fonksiyonu tanımlanır. Dil yapıları matematiksel nesnelere 'gönderimlenir' (denote).

► Temel Kavramlar

- Sözdizimsel Alan (Syntactic Domain): Fonksiyon girdisi — sözdizimsel yapılar
- Anlambilimsel Alan (Semantic Domain): Fonksiyon çıktısı — matematiksel nesneler
- Program Durumu s : $s = \{ \langle i1, v1 \rangle, \dots, \langle in, vn \rangle \}$ — değişken-değer çiftleri
- $VARMAP(ij, s) = vj$: Durum s 'teki ij değişkeninin değerini döndürür

Denotasyonel Semantik Örnekleri

İkili sayıların denotasyonel semantiği:

$\langle bin_num \rangle \rightarrow '0' \mid '1' \mid \langle bin_num \rangle '0' \mid \langle bin_num \rangle '1'$

$Mbin('0') = 0$

$Mbin('1') = 1$

$Mbin(\langle bin_num \rangle '0') = 2 * Mbin(\langle bin_num \rangle)$

$Mbin(\langle bin_num \rangle '1') = 2 * Mbin(\langle bin_num \rangle) + 1$

Örnek: 110 (ikili) $\rightarrow ?$

$Mbin('110') = 2 * Mbin('11') = 2 * (2 * Mbin('1') + 1) = 2 * (2 + 1) = 6 \checkmark$

Atama deyiminin denotasyonel semantiği:

$Ma(x = E, s) \triangleq \text{if } Me(E, s) == \text{error then error}$

else s' nerede $vj' = Me(E, s)$ if $ij == x$,

$vj' = VARMAP(ij, s)$ aksi halde

While döngüsünün denotasyonel semantiği (özyinelemeli):

$MI(\text{while } B \text{ do } L, s) \triangleq$

if $Mb(B, s) == \text{undef} \rightarrow \text{error}$

if $Mb(B, s) == \text{false} \rightarrow s$ (koşul yanlış, dur)

if $Msl(L, s) == \text{error} \rightarrow \text{error}$

else $MI(\text{while } B \text{ do } L, Msl(L, s))$ (özyinelemeli çağrı)

3.5.3 Aksiyomatik Anlambilim (Axiomatic Semantics)

Hoare (1969) tarafından geliştirilmiştir. Program doğruluğunu ispat etmek için matematiksel mantık (predicate calculus) kullanır. Makine modeli yoktur; anlamlar değişkenler arasındaki

ilişkiler üzerinden tanımlanır.

Kavram	İngilizce	Açıklama
Ön Koşul	Precondition {P}	Deyimden önce geçerli olması gereken mantıksal ifade
Son Koşul	Postcondition {Q}	Deyimden sonra geçerli olması gereken mantıksal ifade
En Zayıf ÖK	Weakest Precondition	Verilen {Q}'yu garantileyen en az kısıtlı {P}
Aksiyom	Axiom	Öncülü olmayan çıkarım kuralı; doğru kabul edilir
Çıkarım Kuralı	Inference Rule	Belirli koşullar altında başka bir ifadenin doğruluğunu çıkarır
Sonuç Kuralı	Rule of Consequence	ÖK güçlenebilir, SK zayıflatılabilir
Döngü Değişmezi	Loop Invariant	Döngü boyunca her zaman doğru olan ifade

Atama Aksiyomu Örnekleri

```
# Atama Aksiyomu: {P} x = E {Q} → P = Q(x→E)

# Q'daki tüm x'ler E ile değiştirilir.

# Örnek 1: a = b/2 - 1 {a < 10}

# b/2 - 1 < 10 → b < 22 → En zayıf ÖK: {b < 22}

# Örnek 2: x = 2*y - 3 {x > 25}

# 2*y - 3 > 25 → y > 14 → En zayıf ÖK: {y > 14}

# Örnek 3: x = x + y - 3 {x > 10}

# x+y-3 > 10 → y > 13-x → En zayıf ÖK: {y > 13-x}
```

While Döngüsü Doğrulama

```
# While döngüsü doğrulama:

# {y <= x} while y <> x do y = y + 1 end {y = x}

# Döngü Değişmezi Bulma:

# 0 iter: {y = x}

# 1 iter: wp(y=y+1, {y=x}) = {y+1=x} = {y=x-1}

# 2 iter: {y=x-2} 3 iter: {y=x-3}

# Örüntü: I = {y <= x}

# 4 Kriter Kontrolü:

# 1. P => I: {y<=x} => {y<=x} ✓
```

2. $\{I \text{ and } B\} S \{I\}$:

$\{y \leq x \text{ and } y < x\} y = y + 1 \{y \leq x\} \rightarrow \{y \{y + 1 \leq x\} \checkmark$

3. $\{I \text{ and } !B\} \Rightarrow Q$:

$\{y \leq x \text{ and } y = x\} \Rightarrow \{y = x\} \checkmark$

4. Sonlanma: y her iterasyonda artar, x sabittir $\checkmark$

► Kısmi ve Tam Doğruluk

- Kısmi Doğruluk (Partial Correctness): Döngü sonlanırsa sonuç doğrudur (sonlanma garanti değil).
- Tam Doğruluk (Total Correctness): Kısmi doğruluk + döngü her zaman sonlanır.

Sınav Soruları ve Cevapları

Soru S1:

Sözdizimi (syntax) ile anlambilim (semantics) arasındaki temel fark nedir? Programlama dillerinde bu iki kavram neden ayrı ayrı incelenir?

CEVAP

Sözdizimi, bir programlama dilinin ifadelerinin BİÇİMİNİ tanımlar. Anlambilim ise bu yapıların ANLAMINI tanımlar. Örneğin Java'da 'while (bool_expr) statement' sözdizimi tanımlar; Boolean ifade true iken gövdenin çalışması ise semantiktir. Ayrı incelenmelerinin nedeni: sözdizimi için BNF gibi evrensel kabul görmüş bir notasyon mevcutken anlambilim için böyle bir standart henüz yoktur. Ayrıca sözdizimi analizi (parsing) ile anlambilim analizi derleyicide farklı aşamalarda gerçekleşir.

Soru S2:

Bir gramer muğlak (ambiguous) olduğunda bu durum neden derleyici tasarımında ciddi bir problem yaratır?

CEVAP

Muğlak gramer, aynı cümle için iki veya daha fazla farklı parse tree üretebilir. Derleyiciler, üretecekleri kodu parse tree'ye bakarak belirler. Farklı parse tree'ler farklı anlam ve farklı makine kodu demektir. Örneğin 'A = B + C * A' ifadesinin iki parse tree'si varsa: birinde toplama önce diğerinde çarpma önce hesaplanır, bu da programın yanlış sonuç üretmesine yol açar.

Soru S3:

BNF ile EBNF arasındaki temel farklar nelerdir? EBNF'nin eklediği üç temel uzantıyı açıklayınız.

CEVAP

BNF, sözdizimi tanımlamanın temel formal yöntemidir. EBNF, tanımlayıcı gücü artırmadan okunabilirliği geliştiren uzantılar ekler:

1. Köşeli Parantez []: Opsiyonel kısımları belirtir. Örn: [else stmt] → else olmayabilir.
2. Süslü Parantez { }: Sıfır veya daha fazla tekrarı gösterir. Örn: {, id} → virgüllü liste.
3. Parantez ile |: Bir grup seçenekten birini belirtir. Örn: (* | / | %).

Soru S4:

Sentezlenmiş (synthesized) ve kalıtılan (inherited) öznitelikler arasındaki fark nedir? Her birinin tipik kullanım senaryosunu açıklayınız.

CEVAP

Sentezlenmiş öznitelikler ALT düğümlerden ÜST düğüme bilgi taşır. Değerleri, düğümün çocuklarının özniteliklerine bağlıdır. Örnek: actual_type — değişkenin tipi sembol tablosundan (aşağıdan) gelir, yukarıya iletilir.

Kalıtılan öznitelikler ÜST düğümden ALT düğüme bilgi taşır. Örnek: expected_type — atama deyiminde sol tarafın tipi, sağ taraftaki expr'e 'miras' bırakılır; bu sayede tip uyumluluğu kontrol edilir.

Soru S5:

Aksiyomatik semantikte 'en zayıf ön koşul (weakest precondition)' kavramını açıklayınız. Deyim ' $x = 3*y + 2$ ' ve son koşul $\{x > 14\}$ için hesaplayınız.

CEVAP

En zayıf ön koşul, verilen son koşulu garantileyen EN AZ KISITLAYICI ön koşuldur.

Atama aksiyomu: $P = Q(x \rightarrow E)$ — son koşuldaki tüm x 'ler E ile değiştirilir.

Hesaplama: Deyim: $x = 3*y + 2$ | Son koşul: $\{x > 14\}$

x yerine $3*y+2$ koy: $3*y + 2 > 14 \rightarrow 3y > 12 \rightarrow y > 4$

En zayıf ön koşul: $\{y > 4\}$

Soru S6:

Operasyonel, denotasyonel ve aksiyomatik semantiği karşılaştırınız. Her birinin güçlü ve zayıf yönleri nelerdir?

CEVAP

Operasyonel: İdeal makinede çalıştırarak anlam belirtir. Güçlü: Kullanıcı/implementor için anlaşılır. Zayıf: Matematiksel temeli yoktur, dairesel tanım riski.

Denotasyonel: Her yapıyı matematiksel nesneye eşler. Güçlü: En titiz ve kapsamlı yöntemdir. Zayıf: Karmaşıktır, dil kullanıcıları için pratik değil.

Aksiyomatik: Formel mantıkla program doğruluğunu ispat eder. Güçlü: Program doğrulama için mükemmel çerçeve. Zayıf: Her deyim türü için aksiyom yazmak zordur.

Soru S7:

Bir programlama dilinde operatör önceliği (operator precedence) ve birleşimi (associativity) gramer kurallarına nasıl yansıtılır?

CEVAP

Operatör Önceliği: Parse tree'de daha aşağıda üretilen operatör daha yüksek önceliğe sahiptir. Ayrı nonterminal katmanları (expr, term, factor) kullanarak farklı operatörler farklı derinliklerde üretilir. Daha derin olan önce hesaplanır. Örn: term (çarpma) her zaman expr (toplama)'dan daha derin $\rightarrow$ çarpma önce yapılır.

Birleşim: Sol özyinelemeli kural (LHS sol başta) sol birleşimi, sağ özyinelemeli kural (LHS sağ sonda) sağ birleşimi belirtir. Örn: $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{term} \rangle \rightarrow$ sol birleşim $\rightarrow (A+B)+C$.

Kitap Bölüm Sonu Soruları ve Çözümleri

Problem 6a:

Örnek 3.2'deki grameri kullanarak ' $A = A * (B + (C * A))$ ' için parse tree ve soldan türetmeyi gösteriniz.

ÇÖZÜM

Örnek 3.2 Grameri:

$\langle \text{assign} \rangle \rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\langle \text{id} \rangle \rightarrow A \mid B \mid C$

$\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle \mid \langle \text{id} \rangle * \langle \text{expr} \rangle \mid (\langle \text{expr} \rangle) \mid \langle \text{id} \rangle$

Soldan Türetme:

$\langle \text{assign} \rangle \Rightarrow \langle \text{id} \rangle = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{expr} \rangle$

$\Rightarrow A = \langle \text{id} \rangle * \langle \text{expr} \rangle$

$\Rightarrow A = A * \langle \text{expr} \rangle$

$\Rightarrow A = A * (\langle \text{expr} \rangle)$

$\Rightarrow A = A * (\langle \text{id} \rangle + \langle \text{expr} \rangle)$

$\Rightarrow A = A * (B + \langle \text{expr} \rangle)$

$\Rightarrow A = A * (B + (\langle \text{expr} \rangle))$

$\Rightarrow A = A * (B + (\langle \text{id} \rangle * \langle \text{expr} \rangle))$

$\Rightarrow A = A * (B + (C * \langle \text{expr} \rangle))$

$\Rightarrow A = A * (B + (C * \langle \text{id} \rangle))$

$\Rightarrow A = A * (B + (C * A))$

Parse Tree (sözlü):

$\langle \text{assign} \rangle$

├ $\langle \text{id} \rangle \rightarrow A$

├ $=$

└ $\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle * \langle \text{expr} \rangle$

├ $\langle \text{id} \rangle \rightarrow A$

├ $*$

└ $\langle \text{expr} \rangle \rightarrow (\langle \text{expr} \rangle)$

└ $\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle + \langle \text{expr} \rangle$

├ $\langle \text{id} \rangle \rightarrow B$

├ $+$

└ $\langle \text{expr} \rangle \rightarrow (\langle \text{expr} \rangle)$

└ $\langle \text{expr} \rangle \rightarrow \langle \text{id} \rangle * \langle \text{expr} \rangle$

$\vdash \langle id \rangle \rightarrow C * \langle id \rangle \rightarrow A$

Problem 8:

Aşağıdaki gramerin muğlak olduğunu ispatlayınız:

$\langle S \rangle \rightarrow \langle A \rangle$

$\langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle \mid \langle id \rangle$

$\langle id \rangle \rightarrow a \mid b \mid c$

ÇÖZÜM

İspat: 'a + b + c' cümlesi için iki farklı parse tree gösterilir.

Parse Tree 1 — sol birleşim: (a + b) + c

$\langle S \rangle \rightarrow \langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle$

$\vdash \langle A \rangle + \langle A \rangle (a + b)$

$\vdash \langle id \rangle \rightarrow c$

Parse Tree 2 — sağ birleşim: a + (b + c)

$\langle S \rangle \rightarrow \langle A \rangle \rightarrow \langle A \rangle + \langle A \rangle$

$\vdash \langle id \rangle \rightarrow a$

$\vdash \langle A \rangle + \langle A \rangle (b + c)$

Aynı cümle için iki farklı parse tree → gramer MUĞLAKTIR. ✓

Ayrıca aynı cümle için iki farklı soldan türetme de yapılabilir.

Problem 13:

n adet 'a' ardından n adet 'b' içeren dizileri üreten gramer yazınız ($n > 0$).

ÇÖZÜM

Gramer:

$\langle S \rangle \rightarrow a \langle S \rangle b \mid a b$

Doğrulama:

$n=1: \langle S \rangle \Rightarrow ab \checkmark$

$n=2: \langle S \rangle \Rightarrow a \langle S \rangle b \Rightarrow a ab b = aabb \checkmark$

$n=3: \langle S \rangle \Rightarrow a a \langle S \rangle b b \Rightarrow aaabbb \checkmark$

$n=4: \Rightarrow aaaabbbb \checkmark$

Parse Tree (aabb):

$\langle S \rangle$

$\vdash a$

$\vdash \langle S \rangle \rightarrow ab$

$\vdash b$

Parse Tree (aaaabbbb):

$\langle S \rangle$

$\vdash a$

$\vdash \langle S \rangle$

$\mid \vdash a$

$\mid \vdash \langle S \rangle$

$\mid \mid \vdash a \vdash \langle S \rangle \rightarrow ab \vdash b$

$\mid \vdash b$

$\vdash b$

Problem 15:

Örnek 3.1'deki BNF'i EBNF'e dönüştürünüz.

ÇÖZÜM

Orijinal BNF (Örnek 3.1):

`<program> → begin <stmt_list> end`

`<stmt_list> → <stmt> | <stmt> ; <stmt_list>`

`<stmt> → <var> = <expression>`

`<var> → A | B | C`

`<expression> → <var> + <var> | <var> - <var> | <var>`

EBNF Dönüşümü:

`<program> → begin <stmt> { ; <stmt> } end`

`<stmt> → <var> = <expression>`

`<var> → A | B | C`

`<expression> → <var> [ (+ | -) <var> ]`

Açıklama:

- `<stmt_list>` kuralı ortadan kalkar; `{ }` ile ifade edilir.

- `<expression>`'daki 3 alternatif `[ ]` ile birleştirilir.

(tek değişken de geçerlidir, operatörlü form opsiyoneldir)

Problem 23a:

En zayıf ön koşulu hesaplayınız:

$$a = 2 * (b - 1) - 1 \{a > 0\}$$

ÇÖZÜM

Atama Aksiyomu: $P = Q(a \rightarrow E)$

Son koşul Q: $\{a > 0\}$

$$E = 2 * (b - 1) - 1$$

Q'daki 'a' yerine E:

$$2*(b-1) - 1 > 0$$

$$2*(b-1) > 1$$

$$2b - 2 > 1$$

$$2b > 3$$

$$b > 1.5$$

En Zayıf Ön Koşul: $\{b > 1.5\}$

$$\text{Kontrol: } b=2 \rightarrow a = 2*(2-1)-1 = 1 > 0 \checkmark$$

$$b=1 \rightarrow a = 2*(1-1)-1 = -1, -1 > 0 \text{ değil } \checkmark$$

Problem 23b:

En zayıf ön koşulu hesaplayınız:

$$b = (c + 10) / 3 \{b > 6\}$$

ÇÖZÜM

Atama Aksiyomu: $P = Q(b \rightarrow E)$

Son koşul Q: $\{b > 6\}$

$$E = (c + 10) / 3$$

Q'daki 'b' yerine E:

$$(c + 10) / 3 > 6$$

$$c + 10 > 18$$

$$c > 8$$

En Zayıf Ön Koşul: $\{c > 8\}$

$$\text{Kontrol: } c=9 \rightarrow b = 19/3 \approx 6.33 > 6 \checkmark$$

$$c=8 \rightarrow b = 18/3 = 6, 6 > 6 \text{ değil } \checkmark$$

Problem 24a:

Dizi için en zayıf ön koşulu hesaplayınız:

$$a = 2*b + 1;$$

$$b = a - 3$$

$$\{b < 0\}$$

ÇÖZÜM

Sondan geriye doğru ilerleriz.

Adım 1 — son deyim: $b = a - 3 \{b < 0\}$

Q'daki b yerine a-3:

$$a - 3 < 0 \rightarrow a < 3$$

Ara sonuç: $\{a < 3\}$

Adım 2 — ilk deyim: $a = 2*b + 1 \{a < 3\}$

Q'daki a yerine $2*b+1$:

$$2*b + 1 < 3 \rightarrow 2b < 2 \rightarrow b < 1$$

En Zayıf Ön Koşul: $\{b < 1\}$

Kontrol: $b=0 \rightarrow a=1; b=1-3=-2 < 0 \checkmark$

$b=1 \rightarrow a=3; b=3-3=0, 0 < 0$ değil $\checkmark$

Problem 24b:

Dizi için en zayıf ön koşulu hesaplayınız:

$$a = 3*(2*b + a);$$

$$b = 2*a - 1$$

$$\{b > 5\}$$

ÇÖZÜM

Adım 1 — son deyim: $b = 2*a - 1$ $\{b > 5\}$

Q'daki b yerine $2*a-1$:

$$2*a - 1 > 5 \rightarrow 2a > 6 \rightarrow a > 3$$

Ara sonuç: $\{a > 3\}$

Adım 2 — ilk deyim: $a = 3*(2*b + a)$ $\{a > 3\}$

Q'daki a yerine $3*(2*b+a)$:

$$3*(2*b + a) > 3 \rightarrow 2b + a > 1 \rightarrow a > 1 - 2b$$

En Zayıf Ön Koşul: $\{a > 1 - 2b\}$

Not: Bu, genel durumda hem a hem b'yi içeren bir koşuldur.

Hızlı Başvuru — Bölüm 3 Özeti

Konu	Anahtar Kavram	Hatırlatıcı
Sözdizimi	Biçim (Form)	BNF ile tanımlanır
Anlambilim	Anlam (Meaning)	Evrensel notasyon yok
Lexeme	En küçük anlamlı birim	index, +, 17, begin
Token	Lexeme kategorisi	identifier, plus_op, int_literal
BNF	Metalanguage	<assign> → <var> = <expr>
Nonterminal	Soyut kategori <>	<expr>, <stmt>, <var>
Terminal	Gerçek sembol	=, +, begin, A
Parse Tree	Hiyerarşik yapı	İç düğüm=nonterminal, yaprak=terminal
Muğlaklık	2+ parse tree	Anlam belirsiz → problem!
Operatör Önceliği	Daha derin = önce	* daha derin → önce hesaplanır
Sol Birleşim	LHS solda	<expr> → <expr> + <term>
EBNF	BNF uzantısı	[] opsiyonel, { } tekrar, () seçim
Öznitelik Grameri	Statik semantik	Tip kontrolü; BNF yetmez
Sentezlenmiş	Aşağıdan yukarı	actual_type → üst düğüme
Kalıtılan	Yukarıdan aşağı	expected_type → alt düğüme
Operasyonel Sem.	İdeal makine	C → ara dil
Denotasyonel Sem.	Matematiksel nesne	Mbin('110') = 6
Aksiomatik Sem.	{P} S {Q}	Program doğruluğu ispatı
Weakest Precond.	P = Q(x→E)	Son koşuldan ön koşula
Loop Invariant	Döngü boyunca doğru	Tam doğruluk ispatı için